



Defence Institute of Advanced Technology

Mid-Sem Evaluation Report

Internship At RCI (DRDO)

Offline Agentic AI Application for SDLC

Guide:

Prof. CRS Kumar, M.B. Sai Charan (Scientist C)

Submitted By:

Abhishek Raj (25-26-15)

1. Synopsis

Modern software engineering increasingly uses LLM-based coding assistants, but these systems often remain isolated from the broader Software Development Lifecycle (SDLC). This project proposes an **Autonomous Software Development Lifecycle (A-SDLC)** framework that coordinates specialized AI agents to analyze requirements, understand existing repositories, plan development tasks, perform localized code modifications, and verify the resulting changes.

To support secure execution in offline and resource-constrained environments, the framework introduces a **Hierarchical Software Knowledge Graph (HSKG)** for repository-aware context retrieval and a **stack-based execution model** for decomposing complex requests into manageable tasks. An iterative **Editor–Reviewer loop**, supported by evaluation and fail-safe termination mechanisms, enables repeated correction and verification while minimizing unnecessary code generation.

2. Literature Review

The transition from simple AI assistants to autonomous SDLC agents is an active area of research, primarily focused on how models retrieve context and execute multi-step logic. Recent surveys indicate that while reasoning capabilities have improved, managing repository-scale context remains a significant challenge. Current systems rely heavily on flat, iterative retrieval methods that frequently inject noisy context when navigating deep structural dependencies. Furthermore, most frameworks default to generative rewriting rather than surgical patching, introducing latency and logic degradation in local deployments.

- **LLM-Based Multi-Agent Systems for Software Engineering: Literature Review, Vision, and the Road Ahead (He, J., Treude, C., & Lo, D., 2025):** This paper provides a comprehensive taxonomy of agentic architectures across SDLC phases. The authors argue for using multiple specialized LLM agents, referred to as LLM-based Multi-Agent Systems, where each agent specializes in a single task. While establishing a baseline for the industry, the paper highlights that these systems currently struggle with long-term planning, memory management, and self-correction, which ultimately causes hallucinations. Memory persistence and contextual limits are identified as primary blockers for enterprise adoption, validating our need for token-efficient memory structures like a project graph.
- **Assistance to Autonomy: A Systematic Literature Review of Agentic AI across the SDLC (Apostolou, S. A., et al., 2026):** Focusing on the transition from simple assistants to fully autonomous loops, this review explores why some SDLC phases are successfully automated while others are not. The authors introduce the *Verifiability Principle*, stating that if a phase is verifiable, it is easier to implement. For example, coding is relatively easy to automate because outputs can be verified, whereas planning and architecture are difficult because there is no single optimal solution to a problem statement. The review details the shift toward goal-oriented execution but notes that current agents lack robust evaluation and verification harnesses for production safety, heavily emphasizing the need for the Reflexion loop and automated testing integration proposed in our architecture.

- **Agentic AI in the Software Development Lifecycle: Architecture, Empirical Evidence, and Operational Mechanics (2026):** This research benchmarks current frameworks on complex, multi-file codebases to map out standard failure rates and provides guidance on architecting A-SDLC systems by keeping humans in the loop for verifiability. The empirical evidence demonstrates that AI agents struggle significantly with maintaining architectural consistency over long sessions. This proves the necessity of persistent state management and isolated surgical editing over standard generative prompting.
- **Codebase-Memory: Tree-Sitter-Based Knowledge Graphs for LLM Code Exploration via MCP (2026):** Understanding an entire repository without exceeding token limits is a major challenge for coding agents. This paper formalizes the "Repository Map" pattern by using Tree-sitter to parse the Abstract Syntax Tree (AST) of the codebase and applying PageRank algorithms to score the importance of cross-file references, creating a prioritized knowledge graph. This allows agents to retrieve highly relevant, k-hop topological summaries of the codebase, drastically reducing token consumption while providing deep structural context—a methodology that heavily influences our HSKG implementation.
- **The Hitchhiker's Guide to Agentic AI: From Foundations to Systems (Roitman, H., 2025):** This foundational text formalizes the core concepts required for autonomous workflows, including agentic loop engineering, harnesses, skills, MCP, the A2A protocol, and state management. It establishes the theoretical blueprint for separating an LLM's raw reasoning from its execution environment. Our project translates these theoretical concepts into a practical, localized LangGraph pipeline constrained for offline hardware.

3. Problem Statement

The primary challenge in adopting agentic AI for software engineering is the reliance of many current systems on cloud-based LLMs and external services, which can be unsuitable for proprietary or restricted codebases. This creates a need for an autonomous SDLC framework that can operate securely within local or offline infrastructure.

However, attempting to deploy current multi-agent architectures on resource-constrained local hardware reveals three critical architectural flaws:

- **The Context Window Bottleneck:** Existing frameworks assume access to cloud-based models with massive context windows. In offline environments, flooding the context window with full repository text leads to severe token degradation, high latency, and logic hallucinations.
- **Absence of Topological Memory:** Standard Retrieval-Augmented Generation (RAG) fetches text based on flat keyword or vector proximity. It lacks a structural understanding of the codebase, frequently failing to retrieve distant structural dependencies (e.g., how a database schema update impacts a frontend controller).
- **Risks of Generative Rewriting:** Conventional LLM-based coding workflows typically rewrite entire code files from scratch to implement minor feature requests. This approach is highly destructive and inefficient for local models, risking the accidental deletion of untouched, working logic.

4. Objectives

To address these gaps, this project targets the following objectives:

- **Design a Hierarchical Software Knowledge Graph (HSKG):** Represent the repository structurally and retrieve only task-relevant subgraphs rather than entire codebases.
- **Develop a Stack-Based Task Decomposition Mechanism:** Convert high-level requirements into ordered, manageable development tasks.
- **Implement Localized Code Editing:** Apply targeted code modifications instead of unnecessary full-file generation.
- **Build an Iterative Editor–Reviewer Loop:** Enable code changes to be evaluated, corrected, and re-evaluated until the required threshold is achieved.
- **Introduce Harness-Based Evaluation:** Monitor agent nodes, iterations, tool interactions, and execution trajectories using multiple evaluation metrics.
- **Implement Fail-Safe Execution:** Use iteration limits, error thresholds, and explicit termination conditions to prevent uncontrolled execution.

5. Approach

The proposed A-SDLC is implemented as a **modular, state-driven workflow with iterative feedback loops**. The workflow is orchestrated using LangGraph and consists of repository analysis, requirement interpretation, task decomposition, localized editing, verification, evaluation, and controlled recovery.

- **Repository Analysis and Manifest Generation:** For an existing project, the **Code Analyzer & Manifest Generator Agent** analyzes the repository and generates a manifest describing the project's technology stack, structure, dependencies, and relevant components. This provides the foundation for repository-aware task planning.
- **Requirements and Planning:** The Requirements and Planning Agent interprets the user's request and converts it into an ordered **task stack**, where each task represents a manageable unit of development work.
- **Hierarchical Software Knowledge Graph (HSKG):** The codebase is modelled as a directed, attributed multigraph $G = (V, E)$, where vertices V represent modules, files, classes, functions, and variables, and edges E represent semantic relationships (e.g., calls, modifies, depends_on). When a user requests a change, the Analyzer node retrieves only the precise topological neighborhood $G_k(m_i)$ required for the task, drastically reducing the token footprint.
- **Diff Patching & Stack-Based Editor Loop:** To eliminate destructive generative rewriting, the workflow relies on localized edit proximity. Instead of rewriting entire files from scratch, the system makes targeted, step-by-step edits:
 - **Code Analyzer & Manifest Generator Agent:** analyzes repository, builds project manifest, identifies structure/dependencies.
 - **Requirements & Planning Agent:** understands user request, decomposes requirements, generates task stack, establishes task priorities/dependencies.
 - **Editor:** retrieves relevant context, performs localized modifications.
 - **Reviewer:** validates changes, produces issue/feedback report

After each Editor–Reviewer iteration, the system checks whether the defined quality threshold has been satisfied. If the threshold is not met, the task is returned to the editing loop with the review feedback. Once the task satisfies the required criteria, it is marked complete and the system proceeds to the next task.

6. Implementation

The execution environment is designed for strict offline operation. The orchestration and state-transition logic are implemented using **LangGraph**, while the LLM nodes are executed locally using **Ollama**, supporting the project's offline execution requirement.

Implementation is divided into two tracks:

1. Demonstration Workflow:

A simplified functional workflow has been developed to demonstrate autonomous task execution and generation of small HTML/JS/CSS applications.

2. Actual Project Architecture:

Development is focused on repository analysis and construction of the HSKG/manifest representation. The Analyzer functionality has been established as the foundation, while optimization and accuracy improvements are ongoing.

A **DeepEval-based evaluation harness** has also been incorporated to monitor the agent workflow. It is intended to evaluate individual nodes and multi-step execution trajectories and to provide quantitative feedback for improving the reliability of the workflow.

7. Preliminary Results

- **Demonstration Workflow:** A functional demonstration workflow has been developed to validate the basic agentic execution pipeline. The prototype accepts a user request and autonomously generates a small HTML/JS/CSS-based application.
- **Repository Analysis:** The foundational Analyzer functionality has been implemented to analyze repository structure and generate a project representation. Current development focuses on improving repository mapping accuracy and ensuring that relevant structural dependencies are correctly captured.
- **Task Planning:** The system can interpret a user request and generate an initial ordered task stack. Current optimization focuses on improving task ordering, dependency awareness, and preventing incomplete or conflicting task decomposition.
- **Current Focus:** The immediate development priority is to achieve a reliable accuracy threshold for repository analysis and task planning before finalizing the Editor and Reviewer stages.

8. Summary

The mid-semester phase has established the theoretical and architectural foundation for an offline A-SDLC framework. The proposed system combines LangGraph-based orchestration, HSKG-based repository context, stack-based task decomposition, localized editing, iterative review, and harness-based evaluation. A functional demonstration workflow and foundational repository-analysis components have been implemented, while the complete autonomous editing and verification pipeline remains under active development.

9. Conclusion

The proposed integration of the **Hierarchical Software Knowledge Graph (HSGG)** and stack-based execution is designed to address the memory and context limitations encountered by local offline LLMs. The current implementation establishes the foundational repository-analysis and task-planning components, while the Editor–Reviewer loop and evaluation mechanisms are being further developed and validated.

10. References

- [1] J. He, C. Treude, and D. Lo, "**LLM-based multi-agent systems for software engineering: Literature review, vision, and the road ahead**," *ACM Trans. Softw. Eng. Methodol.*, vol. 34, no. 2, pp. 1–42, Jan. 2025.
- [2] S. A. Apostolou et al., "**Assistance to autonomy: A systematic literature review of agentic AI across the SDLC**," arXiv preprint arXiv:2601.08451, 2026.
- [3] Anonymous et al., "**Agentic AI in the software development lifecycle: Architecture, empirical evidence, and operational mechanics**," arXiv preprint arXiv:2603.11290, 2026.
- [4] Anonymous et al., "**Codebase-Memory: Tree-Sitter-Based Knowledge Graphs for LLM Code Exploration via MCP**," arXiv preprint arXiv:2603.27277, 2026.'
- [5] H. Roitman, **The Hitchhiker's Guide to Agentic AI: From Foundations to Systems**, 1st ed. New York, NY, USA: TechPress, 2025.
- [6] LangChain, "LangGraph Documentation," 2025. [Online]. Available: <https://langchain-ai.github.io/langgraph/>. [Accessed: Oct. 3, 2026].
- [7] DeepEval, "Guides: AI Agent Evaluation," 2025. [Online]. Available: <https://deepeval.com/guides/guides-ai-agent-evaluation>. [Accessed: Oct. 3, 2026].
- [8] Google Cloud, "What is the Model Context Protocol (MCP)?," 2025. [Online]. Available: <https://cloud.google.com/discover/what-is-model-context-protocol>. [Accessed: Oct. 3, 2026].